USE → HERE: SJMP HERE; ∞ loop (end).

① ADD 2 NUMBER;

```
ORG 0000H.      start
MOV A,30H;      load 1st number
ADD A,31H;      ADD second number
MOV 32H,A;      store the result
END             END.
```

② SUB 2 NUMBER;

```
ORG 0000H
CLR C ;         clear carry before subtraction
MOV A,30H;      load first number
SUBB A,31H;     subtract second number
MOV 32H,A;      store result
END;            END
```

③ MUL 2 NUMBER;

```
ORG 0000H
MOV A,30H;      load 1st Num
MOV B,31H;      load 2nd num
MUL AB ;        A= low byte, B= high byte.
MOV 32H,A;      store low byte
MOV 33H,B;      Store high byte
END.            END.
```

④ DIV 2 NUMBER;

```
ORG0000H        ; start
MOV A,#20H      ; load dividend
MOV B,#04H      ; load divisor
DIV AB          ; A = Quotient, B = remainder
MOV R0,A        ; store quotient
MOV R1,B        ; Store remainder
HERE: SJMP HERE; stop.
```

⑤ Transfer 5 Number block;

```
ORG 0000H       ; start
MOV R0,#30H     ; source starting address
MOV R1,#40H     ; destination
MOV R2,#05H     ; No of bytes to transfer

LOOP:
    MOV A,@R0   Take value from source
    MOV @R1,A;  store in destination

    INC R0      ; next source address
    INC R1      ; next destination.
    DJNZ R2,LOOP; repeat until 5 bytes are done
END.            ; end.
```

⊙ Largest of 5 Numbers:

```
MOV R0, #30H ; R0 points to 1st number
MOV R1, #05 ; counter = 5 numbers
MOV A, @R0 ; Assume 1st number is largest

INC R0 ;
LOOP:
    MOV B, @R0 ; Read next NUMBER
    CJNE A,B, CHECK;
    SJMP NEXT ;

CHECK:
    JC NEXT ; if A<B the B is larger
    MOV A, B ; update largest number

NEXT:
    INC R0 ;
    DJNZ R1, LOOP ;

    MOV 40H, A ; store largest number
HERE: SJMP HERE; END.
```

⊙ UNPACK A PACKED BCD NUMBER:

```
MOV A, 30H ; load packed BCD
ANL A, #0F0H ; mask upper nibble → A0H
SWAP A ; Make it 0AH → upper digit.
MOV 40H, A ; store upper digit
MOV A, 30H ; load packed BCD again
ANL A, #0F ; mask lower nibble → 0A
MOV 41H, A ; store lower digit
HERE: SJMP HERE ; END.
```

To generate a delay of 1ms:

```
ORG 0000H ; start
MAIN:
CALL DELAY ; call 1ms delay
SJMP MAIN ; Repeat forever
;-----------------------------
; 1 ms Delay subroutine
;-----------------------------
DELAY:
MOV R2, #250 ; 250x4 cycles = 1000 us = 1ms
HERE:
DJNZ R2, HERE ;
RET ;
END ; END.
```

(a) TURN ON / OFF LED: at P1.0 & switch at P2.7

```
MOV P2, #0FFH ; make port 2 i/p
MOV P1, #00H ; make port 1 o/p
MAIN:
JB P2.7, LED-ON ; if P2.7 = 1 → jump to LED-ON
LED_OFF:
CLR P1.0 ; LED OFF
SJMP MAIN
LED_ON:
SETB P1.0 ; LEN ON
SJMP MAIN
```

① SSD → (Seven segment display) interfacing with 8051

→ To display "5" on SSD

```
ORG 0000H
MOV P1, #00H ; clear port
MOV A, #6DH ; hex code of digit 5
MOV P1, A ; send to SSD
HERE: SJMP HERE ; ∞ loop
END ; stop
```

② DC MOTOR interfacing with 8051

① Clockwise Rotation : (CW)

```
MOV P1, #01H ; IN1 = 1 , IN2 = 0 → clockwise
HERE: SJMP HERE
```

② Anticlockwise Rotation : (ACW)

```
MOV P1, #02H; IN1 = 0, IN2 = 1 → anticlockwise
HERE: SJMP HERE
```

③ Clockwise → stop → Anticlockwise (Both):

```
ORG 0000H
; ---- clockwise ----
MOV P1, #01H ; CW
ACALL DELAY

; ---- Stop ----
MOV P1, #00H; Motor OFF
ACALL DELAY

; ---- ACW ----
MOV P1, #02H ; ACW
ACALL DELAY

HERE: SJMP HERE

; ------ Simple delay -----
DELAY : MOV R7, #255
WAIT : DJNZ R7, WAIT
RET
END
```

STEPPERMOTOR Interfacing with 8051 (Both CW & ACW)

```
ORG 0000H          → MOV P2 #00H; clear port
; — — — — CW — — —
MOV P2, #88H H;
ACALL DELAY
MOV P2, #44H;
ACALL DELAY
MOV P2, #22H;
ACALL DELAY
MOV P2, #11H;
ACALL DELAY

; — — — — stop — — —
  MOV P2, #00H
  ACALL DELAY

; — — — ACW — — —
MOV P2, #11H;
ACALL DELAY
MOV P2, #22H
ACALL DELAY
MOV P2, #44H
ACALL DELAY
MOV P2, #88H
ACALL DELAY

HERE; SJMP HERE

; — — DELAY — — —
DELAY: MOV R7, #255
WAIT : DJNZ R7, WATT
       RET

END ; end
```

⑥ LCD :

```
ORG   0000H
;-----LCD initialization ----
MOV P2 , # 38H ; 8-bit , 2 lines
ACALL  CMD
MOV P2 ; # 0EH; display ON
ACALL  CMD
MOV P2;#01H  ; display clear
ACALL  CMD
MOV  P2 , #06H ; auto increment
ACALL  CMD
MOV  P2, #80H ; cursor at first line
ACALL  CMD

; - - - -  Display 'Hi' - - - -
MOV P2, #'H'
ACALL  DATA
MOV P2,#'I'
ACALL  DATA

HERE : SJMP  HERE

; - - - -  Send command - - - -
CMD     CLR  P3.0      ; RS = 0
        CLR  P3.1      ; RW = 0
        SETB P3.2      ; EN = 1
        ACALL  DELAY ;
        CLR  P3.2  ; EN = 0
        RET

; - - - -  SEND DATA - - - -
DATA :  SETB  P3.0  ; RS = 1
        CLR P3.1    ; RW = 0
        SETB P3.2   ; EN = 1
        ACALL DELAY
        CLR  P3.2  ; EN = 0
        RET
```

```
;- - - - SMALL DELAY - - -
DELAY : MOV R7, #255
WAIT : DJNZ R7, WAIT
       RET

END
```

⑩ HEX KEYPAD with 8051;

```
ORG 0000H
MAIN:
    MOV P1, #0F0H ; upper nibble i/p (cols),
                    lower nibble o/p (rows)
;- - - - ROW 0 - - -
ROW 0:
    MOV P1,#0FEH ;      Row 0 = 0
    JB P1.4, ROW1 ; if Co = 0 → key 1 pressed.
    MOV A, #'1'
    SJMP DONE
;- - - - ROW 1 - - -
ROW 1:
    MOV P1, #0FDH;
    JB P1.4, ROW2;
    MOV A, #'2';
    SJMP DONE.
;- - - - ROW 2 - - -
ROW 2:
    MOV P1, #0FBH
    JB P1.4, ROW3;
    MOV A, #'7';
    SJMP DONE
;- - - - ROW 3 - - -
ROW 3:
    MOV P1; #0F7H
    JB P1.4, ROW 0
    MOV A, #'E'
    SJMP DONE
DONE:
    SJMP MAIN
END.
```

⑥ DC Motor direction control using switch (P2.0):

```
        ORG  0000H
MAIN:
        JB   P2.0, CW ; If P2.0=1 → Jump to CW

ACW:                  ; anticlockwise
        CLR  P1.0;
        SET  P1.1;
        SJMP MAIN

CW:                   ; clockwise
        SET  P1.0
        CLR  P1.1
        SJMP MAIN

        END.
```